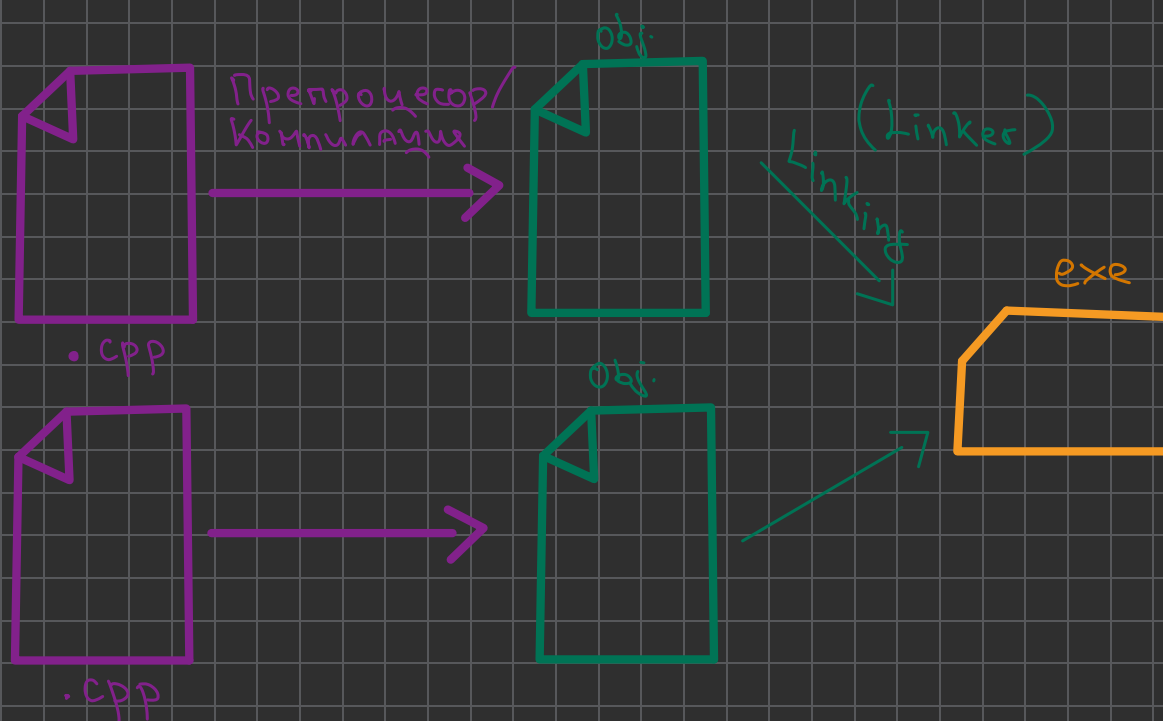


Разделна компилация



#include "file.txt" - заместя режа със съдържанието на file.txt

Защо пишем #pragma once?

- Ставаме X.h в A.h и в B.h
- Ставаме A.h и B.h в C.h
- => в C.h има X.h 2 пъти
- => проблем

Защо не include-ме .cpp?

Защо ползваме разделна комп-я?

- когато направим промяна по 1 файл да не е нужно да комп-ме и другите

Предефиниране на оператори

3 вида оператори:

- унарни $++$, $--$, $-$, $+$
- бинарни $+$, $-$, $*$, $/$
- тернарни $?$

2 Начина да предеф-ме:

I-и: Като вътрешни ф-я:

- Променка лебид арг-т и по 2 аргумента
- Викаме от обекта

Пример: $++$, $--$, $*$, $=$

II-и: Външни ф-я

- Према 2 арг-та
- не ги променя
- връща нов об-т

Пример: $+$, $-$, $*$, $!$, $=$

Операторите се х-т с:

1) Асоциативност: $+$, $-$, $/$, $*$
левоасоциативни, $((a \& b) \& c)$
десноасоциативни $(a \& (b \& c))$

2) Приоритет $a \& b @ c$

3) Поз-я на оператора спрямо аргумента: - суфиксни, $a+b$
инфиксни, $a+b$
префиксни, $++a$

Правила:

! Някои оператори трябва да са тлен-функции!

L $=$, $[]$, $()$, $->$

! Някои оператори не могат да се предефинират!

L $::$, $.$, тернарен оп-р

! Не можем да създадем нови оп-ри!

! Не можем да предеф-ме асоц-т, приор-т, поз-я спрямо арг-те!

! → да връща обект като реф-я / ух-л / копие

! Ако предеф-ме $&l$, $!!$ губим мързеливото оценяване!

Предефиниране на оператори за поток:

< <

→ Трябва да са външни, за да може потокът да е отляво!

→ По 2, за да chain-ме!

→ $cout << z << y;$
cout

> >

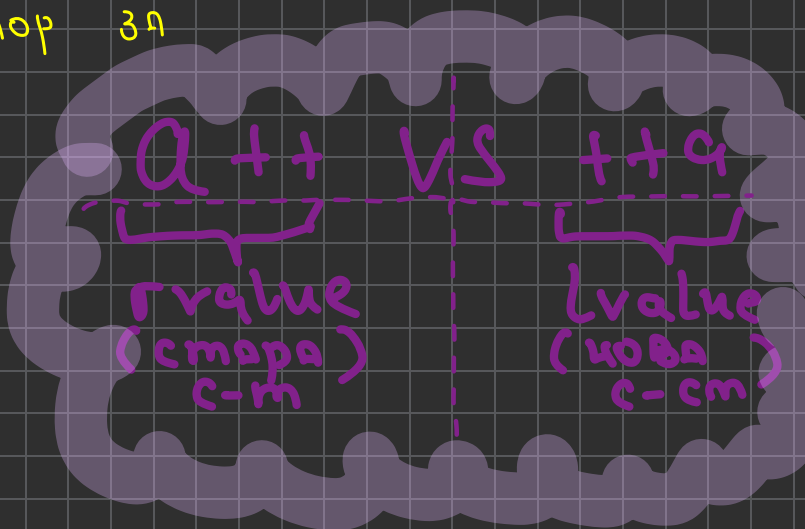
→ Приятелски, за да имаме готови данни!

Ор $+$ / $-$ $*$
не правят копие
заради NRVO

Предefinedирање на оператор за инкрементација:

rvalue
a++
стапа c-м
A op++(int x) {
 this → x++;
 return A(x-1);
}
dummy parameter

lvalue
++a
A op++() {
 this → x++;
 return (*this);
}



Приетелски клас:

Внешен клас, којто има доступ до private / protected член-варијаблите на текучки клас.

! Приетел на мой приетел НЕ ми е приетел!